Importing libraries

In [26]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import my_config

from fredapi import Fred
```

In [27]:
```
fred_key = my_config.FRED_API_KEY
print(f"Key Loaded Succesfully")
```

```
Key Loaded Succesfully
```

In [28]:
```
fred = Fred(api_key=fred_key)
fred
```

Out[28]:
```
<fredapi.fred.Fred at 0x1e482eb5690>
```

In [3]:
```
## previewing of all tables that will be used
```

In [29]:
```
## Searching for specific data

fred_search = fred.search("CPIAUCSL") ## Consumer price index
fred_search.head(4)
```

Out[29]:

series id	id	realtime_start	realtime_end	title	observation_start
CPIAUCSL	CPIAUCSL	2026-01-06	2026-01-06	Consumer Price Index for All Urban Consumers: ...	1947-01-01
RRSFS	RRSFS	2026-01-06	2026-01-06	Advance Real Retail and Food Services Sales	1992-01-01
AMBSLREAL	AMBSLREAL	2026-01-06	2026-01-06	Real St. Louis Adjusted Monetary Base (DISCONT...	1947-01-01
MZMREAL	MZMREAL	2026-01-06	2026-01-06	Real MZM Money Stock (DISCONTINUED)	1959-01-01

In [5]:
```
fred_search = fred.search("CPILFESL") ## Core inflation
fred_search.head(4)
```

Out[5]:

series id	id	realtime_start	realtime_end	title	observation_start	observati
CPILFESL	CPILFESL	2026-01-06	2026-01-06	Consumer Price Index for All Urban Consumers: ...	1957-01-01	2025
CPIAUCSL	CPIAUCSL	2026-01-06	2026-01-06	Consumer Price Index for All Urban Consumers: ...	1947-01-01	2025

In [6]:
```
fred_search = fred.search("FEDFUNDS") ## policy rate
fred_search.shape
```

Out[6]: (1, 15)

In [7]:
```
fred_search = fred.search("DGS10") ## long term rate
fred_search.shape
```

Out[7]: (3, 15)

Pulling RAW data

In [8]:
```
cpi = fred.get_series('CPIAUCSL')
cpi
```

Out[8]:
```
1947-01-01     21.480
1947-02-01     21.620
1947-03-01     22.000
1947-04-01     22.000
1947-05-01     21.950
                ...
2025-07-01    322.132
2025-08-01    323.364
2025-09-01    324.368
2025-10-01        NaN
2025-11-01    325.031
Length: 947, dtype: float64
```

In [9]:
```
core_cpi = fred.get_series('CPILFESL')
core_cpi.head(2)
```

Out[9]:
```
1957-01-01    28.5
1957-02-01    28.6
dtype: float64
```

In [10]:
```
fed_funds = fred.get_series('FEDFUNDS')
fed_funds.tail()
```

Out[10]:
```
2025-08-01    4.33
2025-09-01    4.22
2025-10-01    4.09
2025-11-01    3.88
2025-12-01    3.72
dtype: float64
```

In [11]:
```
treasury = fred.get_series('DGS10')
treasury.head(2)
```

Out[11]:
```
1962-01-02    4.06
1962-01-03    4.03
dtype: float64
```

In [12]:
```
## Combining all the data tables
```

In [13]:
```
df  = pd.concat([cpi,core_cpi,fed_funds,treasury], axis=1)

df.columns = [
```

```
    "headline_cpi",
    "core_cpi",
    "fed_funds_rate",
    "treasury_10y_rate"
]
```

Cleaning data

In [14]:
```
print(cpi.head())
print(core_cpi.head())
print(fed_funds.head())
print(treasury.head())
```

```
1947-01-01    21.48
1947-02-01    21.62
1947-03-01    22.00
1947-04-01    22.00
1947-05-01    21.95
dtype: float64
1957-01-01    28.5
1957-02-01    28.6
1957-03-01    28.7
1957-04-01    28.8
1957-05-01    28.8
dtype: float64
1954-07-01    0.80
1954-08-01    1.22
1954-09-01    1.07
1954-10-01    0.85
1954-11-01    0.83
dtype: float64
1962-01-02    4.06
1962-01-03    4.03
1962-01-04    3.99
1962-01-05    4.02
1962-01-08    4.03
dtype: float64
```

In [15]:
```
df.info
```

Out[15]:
```
<bound method DataFrame.info of             headline_cpi  core_cpi  fed_funds_rate
treasury_10y_rate
1947-01-01         21.48       NaN             NaN                NaN
1947-02-01         21.62       NaN             NaN                NaN
1947-03-01         22.00       NaN             NaN                NaN
1947-04-01         22.00       NaN             NaN                NaN
1947-05-01         21.95       NaN             NaN                NaN
...                  ...       ...             ...                ...
2025-12-29           NaN       NaN             NaN               4.12
2025-12-30           NaN       NaN             NaN               4.14
2025-12-31           NaN       NaN             NaN               4.18
2026-01-01           NaN       NaN             NaN                NaN
2026-01-02           NaN       NaN             NaN               4.19

[17099 rows x 4 columns]>
```

In [16]:
```
## Preveiwing data

df.head(5)
```

Out[16]:

	headline_cpi	core_cpi	fed_funds_rate	treasury_10y_rate
1947-01-01	21.48	NaN	NaN	NaN
1947-02-01	21.62	NaN	NaN	NaN
1947-03-01	22.00	NaN	NaN	NaN
1947-04-01	22.00	NaN	NaN	NaN
1947-05-01	21.95	NaN	NaN	NaN

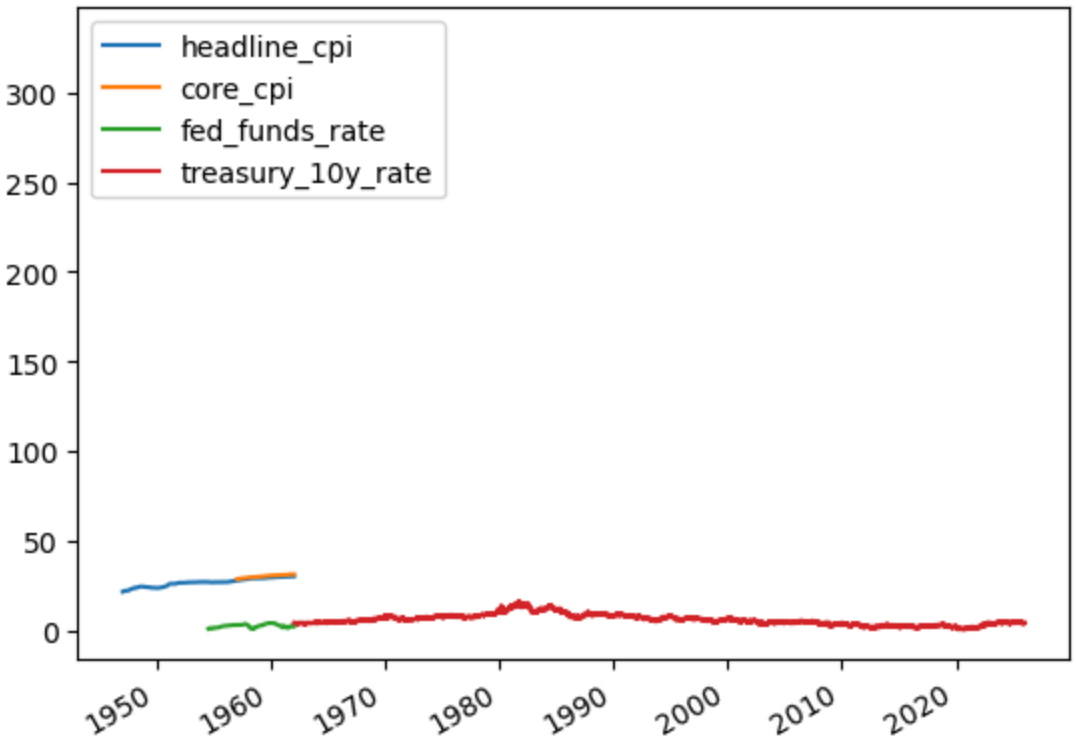
In [17]:
```
df.tail(5)
```

Out[17]:

	headline_cpi	core_cpi	fed_funds_rate	treasury_10y_rate
2025-12-29	NaN	NaN	NaN	4.12
2025-12-30	NaN	NaN	NaN	4.14
2025-12-31	NaN	NaN	NaN	4.18
2026-01-01	NaN	NaN	NaN	NaN
2026-01-02	NaN	NaN	NaN	4.19

In [18]:
```
df.plot() ## using a plot to determine years/frequency of data collection
```

Out[18]:
```
<Axes: >
```



In [ ]:

In [19]:
```
## Confirming datetime indexes to avoid hidden bugs

cpi.index = pd.to_datetime(cpi.index)
core_cpi.index = pd.to_datetime(core_cpi.index)
fed_funds.index = pd.to_datetime(fed_funds.index)
treasury.index = pd.to_datetime(treasury.index)
```

In [20]:
```
treasury.index
```

Out[20]:
```
DatetimeIndex(['1962-01-02', '1962-01-03', '1962-01-04', '1962-01-05',
               '1962-01-08', '1962-01-09', '1962-01-10', '1962-01-11',
               '1962-01-12', '1962-01-15',
               ...
               '2025-12-22', '2025-12-23', '2025-12-24', '2025-12-25',
               '2025-12-26', '2025-12-29', '2025-12-30', '2025-12-31',
               '2026-01-01', '2026-01-02'],
              dtype='datetime64[ns]', length=16699, freq=None)
```

In [21]:
```
## Converting daily treasury yields into monthly averages

treasury_monthly = treasury.resample('MS').mean()

treasury_monthly.tail(5)
```

Out[21]:
```
2025-09-01    4.120476
2025-10-01    4.061818
2025-11-01    4.093889
2025-12-01    4.143182
2026-01-01    4.190000
Freq: MS, dtype: float64
```

In [22]:
```
##treasury_10y_monthly.index = treasury_monthly.index.to_period("M").to_timestamp("
```

In [23]:
```
## Combining all series

df = pd.concat([
    cpi,core_cpi,fed_funds,treasury_monthly], axis=1)
```

In [24]:
```
df.tail()
```

Out[24]:

	0	1	2	3
2025-09-01	324.368	330.542	4.22	4.120476
2025-10-01	NaN	NaN	4.09	4.061818
2025-11-01	325.031	331.068	3.88	4.093889
2025-12-01	NaN	NaN	3.72	4.143182
2026-01-01	NaN	NaN	NaN	4.190000

In [25]:
```
## Renaming columns

df.columns = [
    "headline_cpi_index",
    "core_cpi_index",
    "fed_funds_rate",
    "treasury_rate"
]
```

In [26]:
```
## Checking for number of missing values

df.isna().sum()
```

Out[26]:
```
headline_cpi_index      3
core_cpi_index        123
fed_funds_rate         91
treasury_rate         180
dtype: int64
```

In [27]:
```
## removing rows with missing values

df_clean = df.dropna()
df_clean.index.name = 'date'
```

In [28]:
```
df_clean.info()
#df_clean.head()
```

```
df_clean.tail(10)
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 766 entries, 1962-01-01 to 2025-11-01
Data columns (total 4 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   headline_cpi_index  766 non-null    float64
 1   core_cpi_index      766 non-null    float64
 2   fed_funds_rate      766 non-null    float64
 3   treasury_rate       766 non-null    float64
dtypes: float64(4)
memory usage: 29.9 KB
```

Out[28]:

date	headline_cpi_index	core_cpi_index	fed_funds_rate	treasury_rate
2025-01-01	319.086	324.739	4.33	4.629048
2025-02-01	319.775	325.475	4.33	4.451053
2025-03-01	319.615	325.659	4.33	4.280476
2025-04-01	320.321	326.430	4.33	4.279048
2025-05-01	320.580	326.854	4.33	4.423810
2025-06-01	321.500	327.600	4.33	4.383500
2025-07-01	322.132	328.656	4.33	4.391818
2025-08-01	323.364	329.793	4.33	4.264762
2025-09-01	324.368	330.542	4.22	4.120476
2025-11-01	325.031	331.068	3.88	4.093889

In [29]:
```
df_clean.describe
```

Out[29]:
```
<bound method NDFrame.describe of             headline_cpi_index  core_cpi_index
fed_funds_rate  treasury_rate
date
1962-01-01              30.040          31.200            2.15       4.083182
1962-02-01              30.110          31.200            2.37       4.039444
1962-03-01              30.170          31.300            2.85       3.930455
1962-04-01              30.210          31.300            2.78       3.843000
1962-05-01              30.240          31.400            2.36       3.873636
...                        ...             ...             ...            ...
2025-06-01             321.500         327.600            4.33       4.383500
2025-07-01             322.132         328.656            4.33       4.391818
2025-08-01             323.364         329.793            4.33       4.264762
2025-09-01             324.368         330.542            4.22       4.120476
2025-11-01             325.031         331.068            3.88       4.093889

[766 rows x 4 columns]>
```

Analysis Based on Objectives

1. Is Inflation pressure easing or persistent?
2. Which inflation measure better reflects ongoing cost pressure?
3. Combine inflation and interest rate indicators into a financial stress framework.
4. Analyze how interest rates respond to inflation dynamics over time to understand monetary policy reactions.
5. Identify signals that suggest continued economic or financial stress

In [30]:
```
## OBJ 1: Is Inflation pressure easing or persistent?


df.columns
```

Out[30]:
```
Index(['headline_cpi_index', 'core_cpi_index', 'fed_funds_rate',
       'treasury_rate'],
      dtype='object')
```

In [31]:
```
df_clean['headline_cpi_index'].isna().sum()
```

Out[31]:
```
0
```

In [32]:
```
df['core_cpi_index'].isna().sum()
```

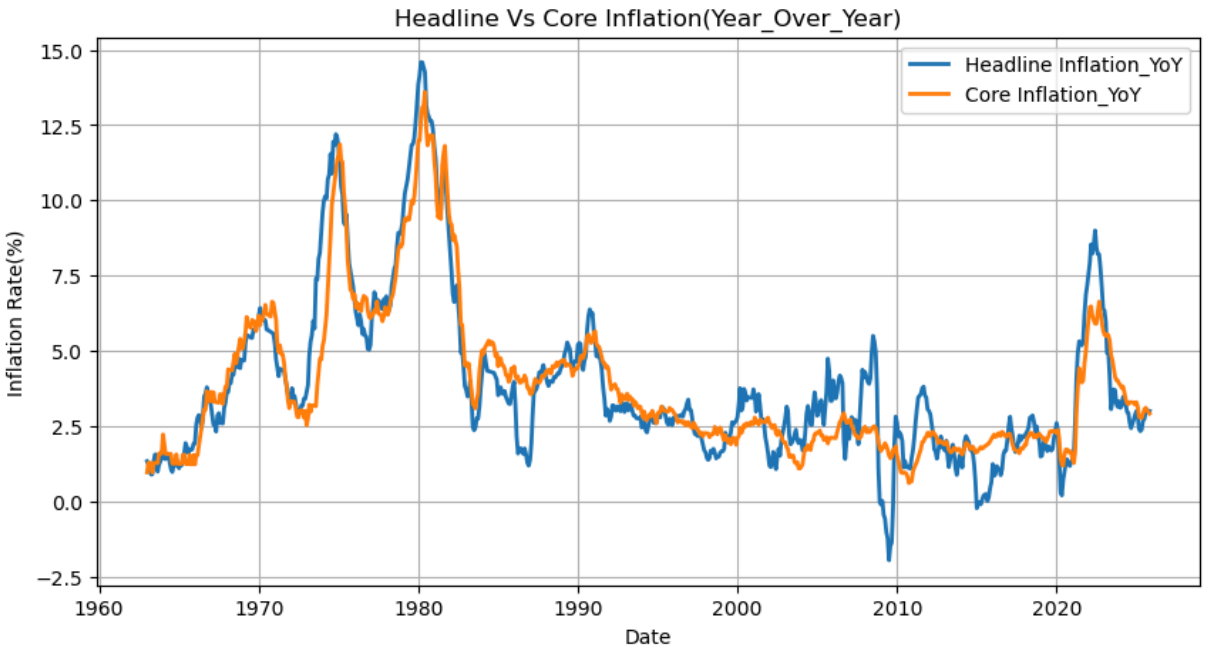
Out[32]:
```
123
```

In [33]:
```
## Removing the year-over-year inflation rates

df_inflation = df_clean.copy()

df_inflation["headline_inflation_yoy"] = (
    df_inflation["headline_cpi_index"].pct_change(12) * 100
)

df_inflation["core_inflation_yoy"] = (
    df_inflation["core_cpi_index"].pct_change(12) * 100
)
```

In [34]:
```
## dropping rows where inflation could not be calculated

df_inflation = df_inflation.dropna()
```

In [35]:
```
## Visualizing headline_inflation vs core_inflation

plt.figure(figsize=(10,5))

plt.plot(
    df_inflation.index,
    df_inflation["headline_inflation_yoy"],
    label = "Headline Inflation_YoY",
    linewidth = 2


)
```

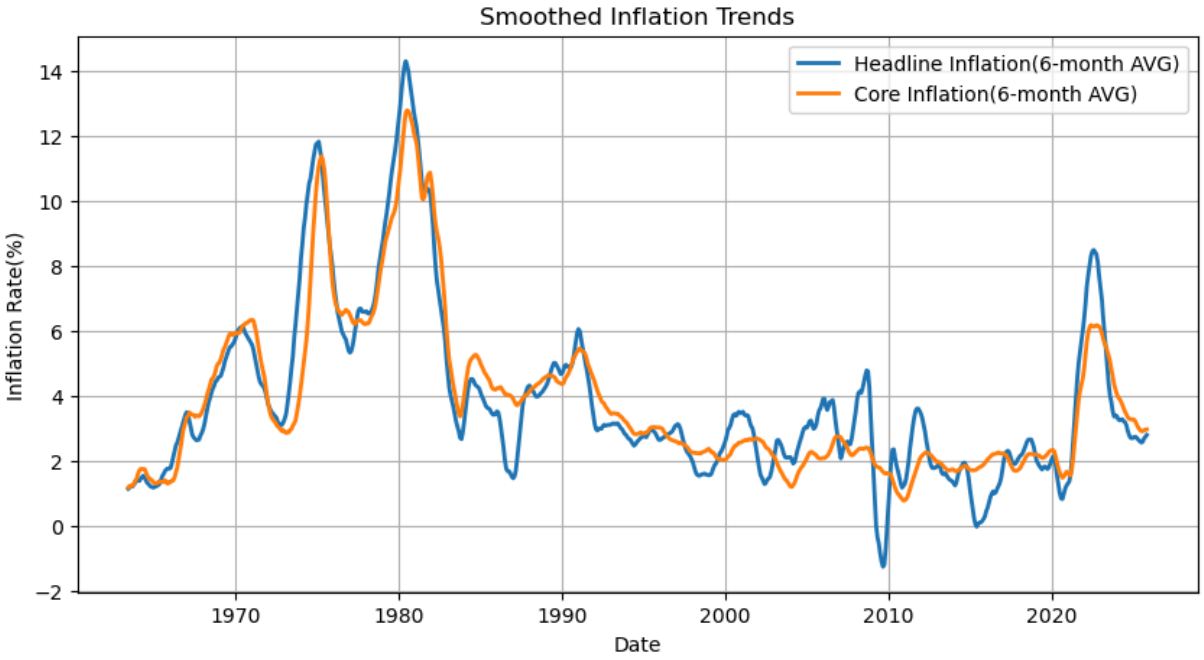
```
plt.plot(
    df_inflation.index,
    df_inflation["core_inflation_yoy"],
    label = "Core Inflation_YoY",
    linewidth = 2
)

plt.title("Headline Vs Core Inflation(Year_Over_Year)")
plt.xlabel("Date")
plt.ylabel("Inflation Rate(%)")
plt.legend()
plt.grid(True)

plt.show()
```



In [36]:
```
## Plottting a 6-Month rolling average

df_inflation["headline_inflation_6m_avg"] = (
    df_inflation["headline_inflation_yoy"].rolling(6).mean()
)

df_inflation["core_inflation_6m_avg"] = (
    df_inflation["core_inflation_yoy"].rolling(6).mean()
)
```

In [37]:
```
plt.figure(figsize=(10,5))

plt.plot(
    df_inflation.index,
    df_inflation["headline_inflation_6m_avg"],
    label = "Headline Inflation(6-month AVG)",
    linewidth = 2


)
```

```
plt.plot(
    df_inflation.index,
    df_inflation["core_inflation_6m_avg"],
    label = "Core Inflation(6-month AVG)",
    linewidth = 2
)

plt.title("Smoothed Inflation Trends")
plt.xlabel("Date")
plt.ylabel("Inflation Rate(%)")
plt.legend()
plt.grid(True)

plt.show()
```



In [38]:
```
# Obj 2: Assesing how changes in interest rates transmit into inflation trends time
```

"headline_cpi_index", "core_cpi_index", "fed_funds_rate", "treasury_rate"

In [39]:
```
## Previewing data to be used for analysis

#df['headline_cpi_index'].head()
df['headline_cpi_index'].describe()
```

Out[39]:
```
count    946.000000
mean     123.634661
std       89.018562
min       21.480000
25%       32.775000
50%      109.550000
75%      199.375000
max      325.031000
Name: headline_cpi_index, dtype: float64
```

In [40]:
```
#df['core_cpi_index'].tail(5)
df['core_cpi_index'].describe()
```

Out[40]:
```
count    826.000000
mean     141.619644
std       88.233855
min       28.500000
25%       47.675000
50%      141.550000
75%      216.286000
max      331.068000
Name: core_cpi_index, dtype: float64
```

In [41]:
```
df['fed_funds_rate'].tail(5)
df['fed_funds_rate'].describe()
```

Out[41]:
```
count    858.000000
mean       4.603800
std        3.542912
min        0.050000
25%        1.910000
50%        4.325000
75%        6.127500
max       19.100000
Name: fed_funds_rate, dtype: float64
```

In [42]:
```
## Copying cleaned data

df_obj2 = df_clean.copy()

# Calculating month-on-month change in interest rate
df_obj2['rate_change'] = df_obj2['fed_funds_rate'].diff()
```

In [43]:
```
df_obj2[['rate_change','fed_funds_rate']].describe()
```

Out[43]:

	rate_change	fed_funds_rate
count	765.000000	766.000000
mean	0.002261	4.861880
std	0.497486	3.652516
min	-6.630000	0.050000
25%	-0.060000	1.985000
50%	0.010000	4.805000
75%	0.110000	6.560000
max	3.060000	19.100000

Preparing inflation changes

In [44]:
```
df_obj2['headline_inflation_MoM'] = df_obj2['headline_cpi_index'].pct_change()*100
df_obj2['core_inflation_MoM'] =df_obj2['core_cpi_index'].pct_change()*100
```

In [45]:
```
## Previewing changes
df_obj2[['headline_inflation_MoM','core_inflation_MoM']].isna().sum()
```

Out[45]:
```
headline_inflation_MoM    1
core_inflation_MoM        1
dtype: int64
```

In [46]:
```
## Removing N/A values

df_obj2 = df_obj2[['rate_change','headline_inflation_MoM','core_inflation_MoM']].dr
```

In [47]:
```
df_obj2.isna().sum()
```

Out[47]:
```
rate_change               0
headline_inflation_MoM    0
core_inflation_MoM        0
dtype: int64
```

In [48]:
```
## Lag Analysis

# creating lagged inflation variables

for lag in[3,6,12]:
    df_obj2[f'headline_inflation_lag_{lag}'] = df_obj2['headline_inflation_MoM'].sh
    df_obj2[f'core_inflation_lag_{lag}'] = df_obj2['core_inflation_MoM'].shift(-lag
```

In [49]:
```
# Dropping N/A rows created due to lag
df_lagged = df_obj2.dropna()
df_lagged.isna().sum()
```

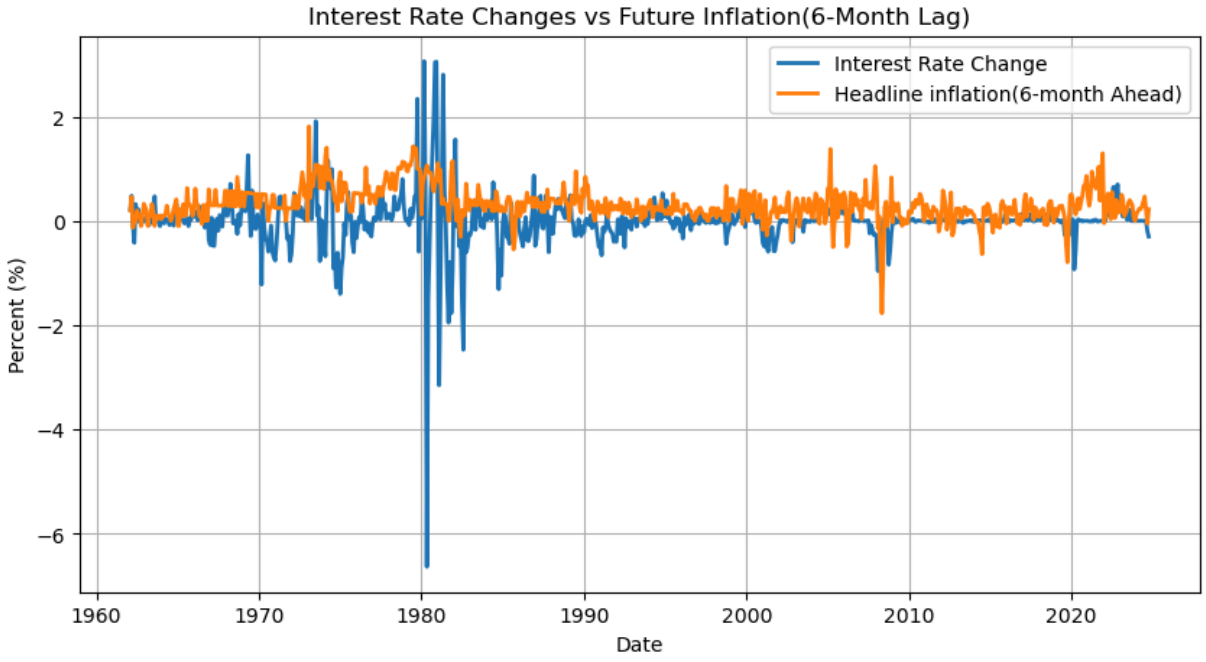
Out[49]:
```
rate_change                 0
headline_inflation_MoM      0
core_inflation_MoM          0
headline_inflation_lag_3    0
core_inflation_lag_3        0
headline_inflation_lag_6    0
core_inflation_lag_6        0
headline_inflation_lag_12   0
core_inflation_lag_12       0
dtype: int64
```

In [50]:
```
## Correlation Analysis between rate changes and future inflation

correlations = {}

for lag in[3,6,12]:
    correlations[f'Headline CPI lag{lag}'] = df_lagged['rate_change'].corr(
            df_lagged[f'headline_inflation_lag_{lag}']

    )
    correlations[f'Core CPI lag{lag}'] = df_lagged['rate_change'].corr(
             df_lagged[f'core_inflation_lag_{lag}']
```

```
    )

correlations
```

Out[50]:
```
{'Headline CPI lag3': 0.1264197334998538,
 'Core CPI lag3': 0.12408065499875118,
 'Headline CPI lag6': 0.07437890717972832,
 'Core CPI lag6': 0.035840188935610424,
 'Headline CPI lag12': 0.11323816596851718,
 'Core CPI lag12': 0.10413300850813408}
```

In [51]:
```
## Interest Rate changes vs future Inflation


plt.figure(figsize=(10,5))

plt.plot(
    df_lagged.index,
    df_lagged['rate_change'],
    label='Interest Rate Change',
    linewidth=2
)

plt.plot(
    df_lagged.index,
    df_lagged['headline_inflation_lag_6'],
    label='Headline inflation(6-month Ahead)',
    linewidth=2
)

plt.title('Interest Rate Changes vs Future Inflation(6-Month Lag)')
plt.xlabel('Date')
plt.ylabel('Percent (%)')
plt.legend()
plt.grid(True)

plt.show()
```



In [52]:
```
# OBJ 3: Combine inflation and interest rate indicators into a financial stress fra
```

In [53]:
```
## Copy df_cleaned

df_obj3 = df_clean.copy()
df_obj3.head()
```

Out[53]:

date	headline_cpi_index	core_cpi_index	fed_funds_rate	treasury_rate
1962-01-01	30.04	31.2	2.15	4.083182
1962-02-01	30.11	31.2	2.37	4.039444
1962-03-01	30.17	31.3	2.85	3.930455
1962-04-01	30.21	31.3	2.78	3.843000
1962-05-01	30.24	31.4	2.36	3.873636

In [54]:
```
##YoY headline inflation(cost pressure)

df_obj3['headline_inflation_YoY'] = (
    df_obj3['headline_cpi_index'].pct_change(12)*100
)

## YoY core inflation
df_obj3['core_inflation_YoY'] = (
    df_obj3['core_cpi_index'].pct_change(12)*100
)
```

In [55]:
```
## keeping interest rates as levels not changes
df_obj3['interest_rate_level'] = df_obj3['fed_funds_rate']
```

In [56]:
```
## Dropping rows with incomplete stress components

df_obj3 = df_obj3[[
    'headline_inflation_YoY','core_inflation_YoY','interest_rate_level'
]].dropna()
```

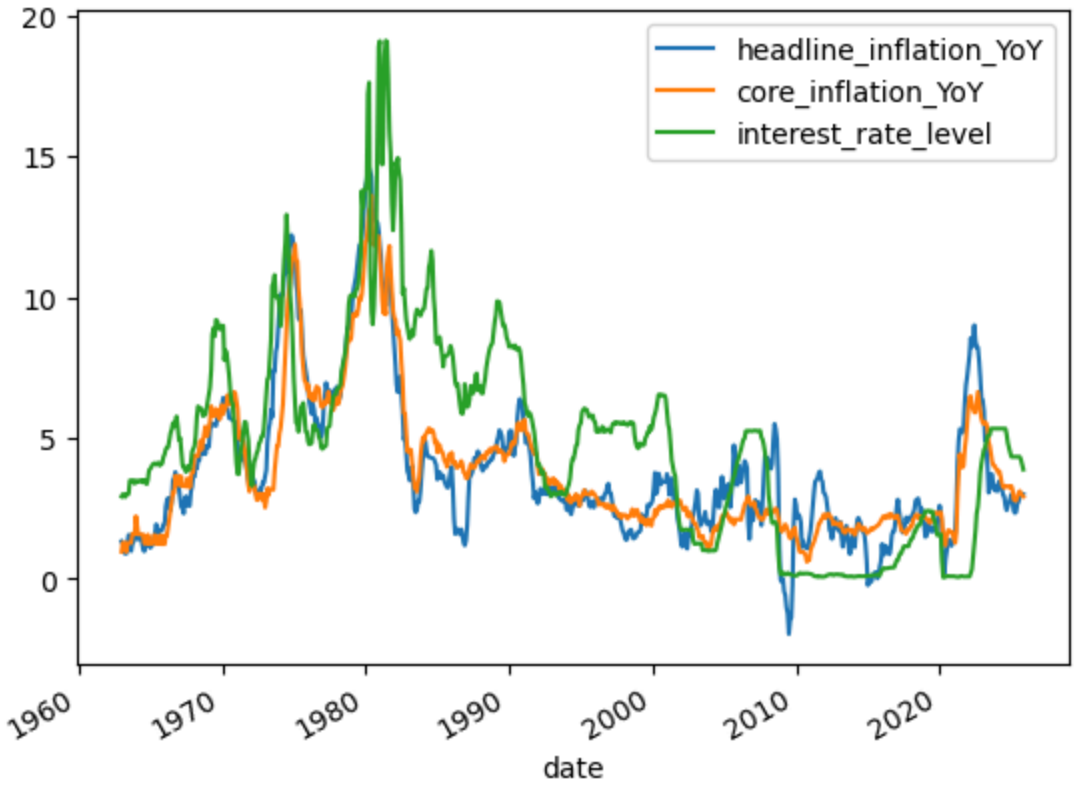
In [57]:
```
df_obj3.tail()
```

Out[57]:

date	headline_inflation_YoY	core_inflation_YoY	interest_rate_level
2025-06-01	2.672683	2.907870	4.33
2025-07-01	2.731801	3.048603	4.33
2025-08-01	2.939220	3.112191	4.33
2025-09-01	3.022700	3.025543	4.22
2025-11-01	3.000025	2.915869	3.88

In [58]:
```
df_obj3.plot()
```

Out[58]:
```
<Axes: xlabel='date'>
```

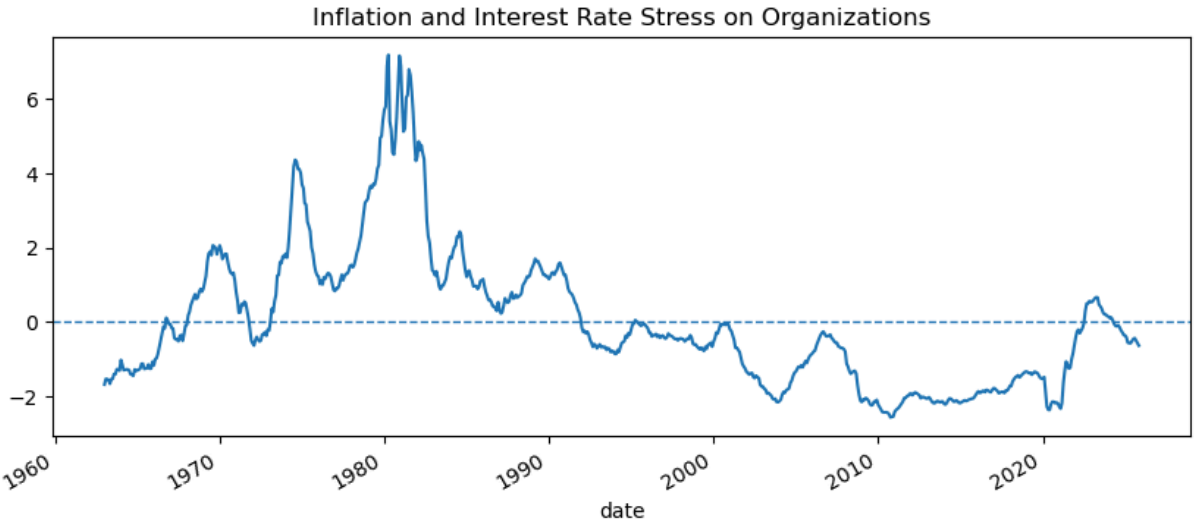


In [59]:
```
##Normalize components because inflation(%) and interest_rates havve different scal

from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()

df_obj3[['inflation_z','rate_z']] = scaler.fit_transform(
    df_obj3[['core_inflation_YoY','interest_rate_level']]

)
```

In [60]:
```
## Stress index(combined cost + financial stress index)

df_obj3['financial_stress_index'] = (
    df_obj3['inflation_z'] + df_obj3['rate_z']
)
```

In [61]:
```
## Stress overtime

df_obj3['financial_stress_index'].plot(
    figsize=(10,4),
    title = "Inflation and Interest Rate Stress on Organizations"
)

plt.axhline(0,linestyle='--',linewidth=1)
plt.show()
```



In [62]:
```
#Obj4: Analyze how interest rates respond to inflation over time.
```

In [63]:
```
df_Obj4 = df_clean.copy()
```

In [64]:
```
df_Obj4.columns
```

Out[64]:
```
Index(['headline_cpi_index', 'core_cpi_index', 'fed_funds_rate',
       'treasury_rate'],
      dtype='object')
```

In [65]:
```
## Calculating YoY inflation(Standard policy review)
df_Obj4['headline_cpi_yoy'] = df_Obj4['headline_cpi_index'].pct_change(12) * 100
df_Obj4['core_cpi_yoy'] = df_Obj4['core_cpi_index'].pct_change(12) * 100
```

```
## Keeping fed funds at the percentage level
df_Obj4 = df_Obj4[['headline_cpi_yoy','core_cpi_yoy','fed_funds_rate']]
```

In [66]:
```
df_Obj4.describe()
```

Out[66]:

	headline_cpi_yoy	core_cpi_yoy	fed_funds_rate
count	754.000000	754.000000	766.000000
mean	3.866225	3.828163	4.861880
std	2.801303	2.492901	3.652516
min	-1.958761	0.602718	0.050000
25%	2.026472	2.115603	1.985000
50%	3.089712	3.006809	4.805000
75%	4.737477	4.817707	6.560000
max	14.592275	13.604488	19.100000

In [67]:
```
df_Obj4.isna().sum()
```

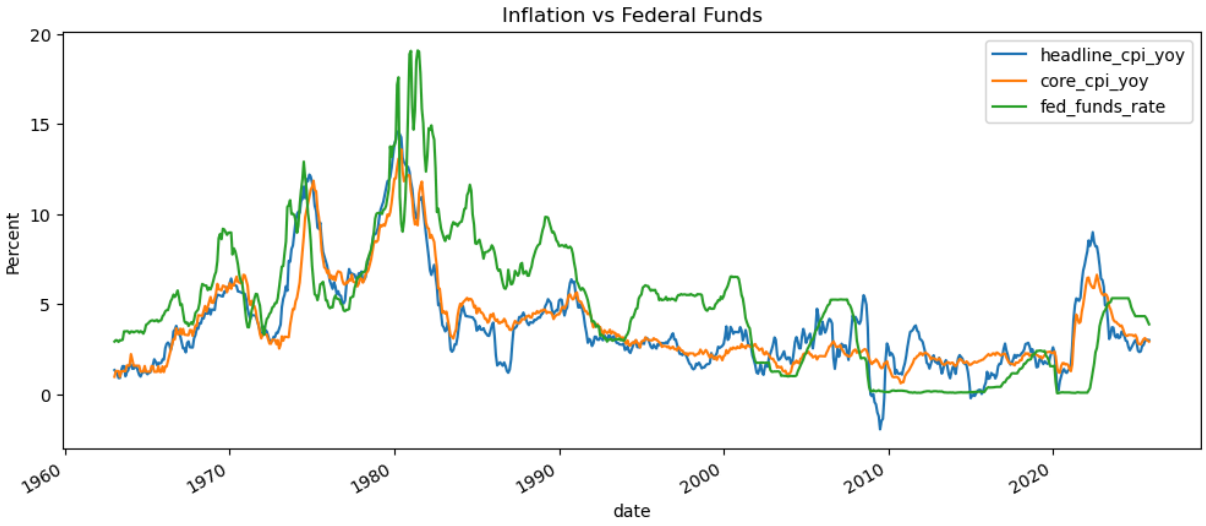
Out[67]:
```
headline_cpi_yoy    12
core_cpi_yoy        12
fed_funds_rate       0
dtype: int64
```

In [68]:
```
# dropping rows where calculations are not available
df_Obj4 = df_Obj4.dropna()
```

In [69]:
```
df_Obj4.isna().sum()
```

Out[69]:
```
headline_cpi_yoy    0
core_cpi_yoy        0
fed_funds_rate      0
dtype: int64
```

In [70]:
```
## Visual Comparison

df_Obj4.plot(
    figsize=(12,5),
    title='Inflation vs Federal Funds'
)

plt.ylabel('Percent')
plt.show()
```



In [71]:
```
## Lag Analysis(does policy react with delay)

#creating lag inflation variables(3,6,12 months)

for lag in [3,6,12]:
    df_Obj4[f'headline_lag{lag}'] = df_Obj4['headline_cpi_yoy'].shift(lag)
    df_Obj4[f'core_lag{lag}'] = df_Obj4['core_cpi_yoy'].shift(lag)

## df_lag = df_Obj4.isna().sum()
```

In [72]:
```
df_lag = df_Obj4.dropna()
```

In [73]:
```
df_lag.columns
```

Out[73]:
```
Index(['headline_cpi_yoy', 'core_cpi_yoy', 'fed_funds_rate', 'headline_lag3',
       'core_lag3', 'headline_lag6', 'core_lag6', 'headline_lag12',
       'core_lag12'],
      dtype='object')
```

In [74]:
```
## Relationship check

correlation_results = df_lag[
    ['fed_funds_rate',
    'headline_lag3','core_lag3', 'headline_lag6', 'core_lag6', 'headline_lag12',
       'core_lag12']
].corr()

correlation_results['fed_funds_rate'].sort_values(ascending=False)
```

Out[74]:
```
fed_funds_rate    1.000000
core_lag3         0.732903
core_lag6         0.728317
core_lag12        0.704274
headline_lag6     0.697237
headline_lag3     0.696244
headline_lag12    0.674705
Name: fed_funds_rate, dtype: float64
```

In [75]:
```
plt.figure(figsize=(10,5))

sns. heatmap(
    correlation_results,
    annot=True,
    cmap='coolwarm',
    fmt=".2f",
    linewidth=0.5

)

plt.title('Federal Rate Reaction: Correlation of Rates vs Lagged Inflation')
plt.ylabel('Variables')
plt.xlabel('Variables')

plt.show()
```